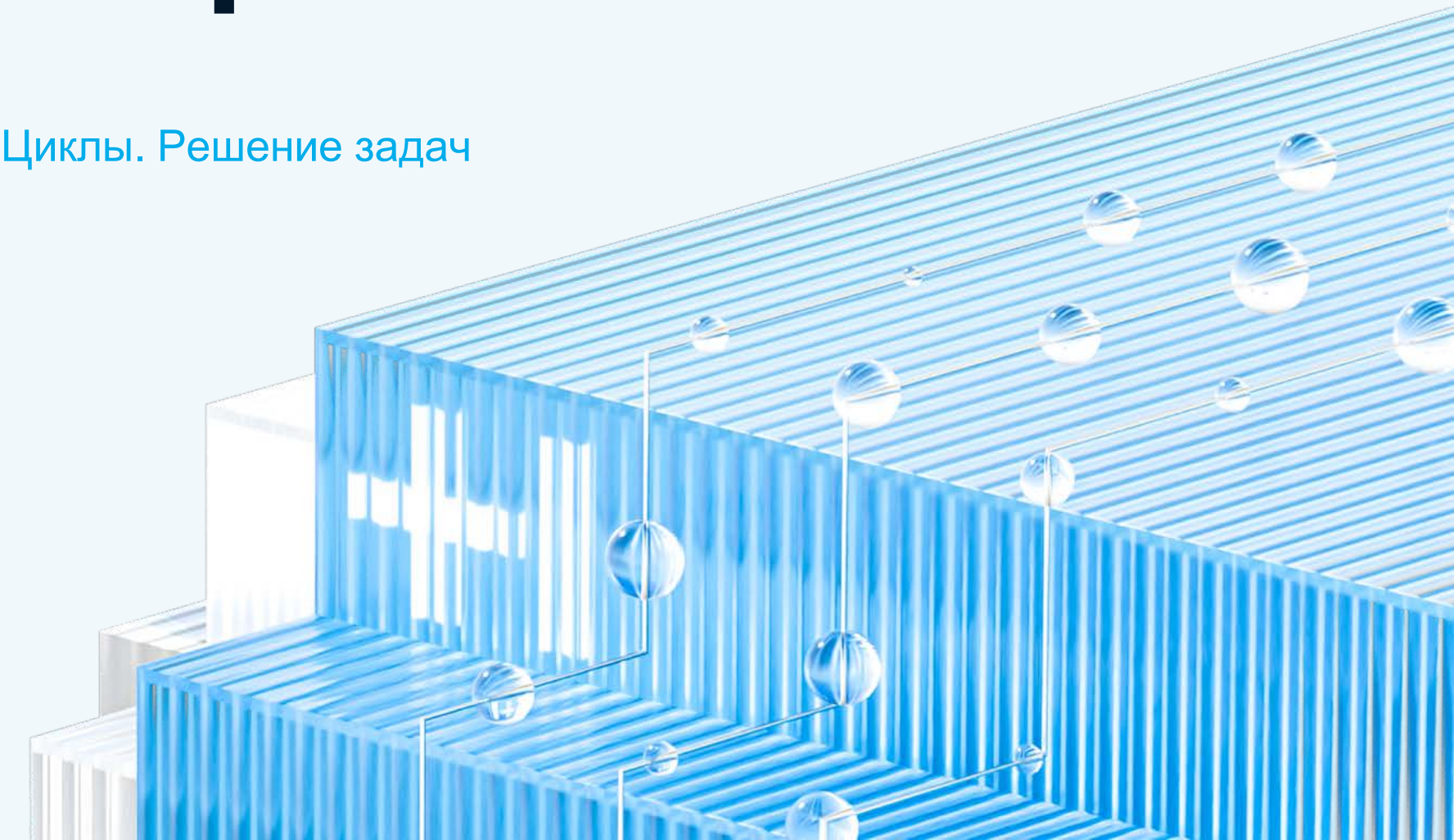
Основы программирования

16.10.2026

Лабораторная работа № 5. Циклы. Решение задач на табуляцию и итерации.



Проклов Роман Сергеевич

ITI

Цели лабораторной работы



1. **Познакомиться** с понятием цикла и его видами
2. **Научиться** использовать циклы while и for в Python и C++
3. **Изучить** операторы break, continue, pass и их применение
4. **Освоить** вложенные циклы и табуляцию функций
5. **Применить** полученные знания для решения практических задач на итерации
6. **Отработать** навыки работы с Git: коммиты, ветвление, push

Формируемые компетенции: ППК-Р1.1, ППК-Р1.4, ППК-Р1.5

План лабораторной работы

Часть 1. Теория: основы циклов

1. Что такое цикл? Виды циклов. Блок-схемы
2. Цикл while в Python и C++
3. Цикл for в Python и C++
4. Сравнение циклов

Часть 2. Теория: управление циклом

5. Оператор break
6. Оператор continue
7. Оператор pass (Python) и пустой оператор (C++)
8. Вложенные циклы

Часть 3. Практика

9. Табуляция функций
10. Задачи на итерации
11. Работа в мини-группах
12. Сдача отчёта и защита



Основы циклов

Цикл: повторение действий

Цикл — то алгоритмическая конструкция, которая позволяет многократно повторять группу действий, пока выполняется некоторое условие.

Виды циклов:

- Цикл с предусловием: while — проверка условия до тела
- Цикл с постусловием: do-while (C++) — проверка после тела
- Цикл с параметром: for — известное число итераций

Пример из жизни:

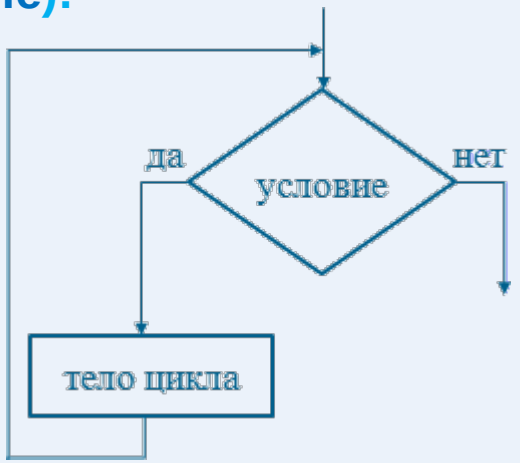
- Пока есть грязная посуда, мыть следующую тарелку
- Для каждого студента в группе вывести его имя

Роль в программировании:

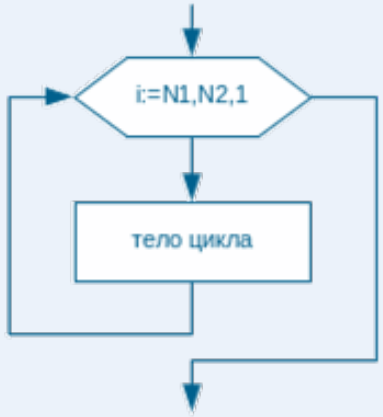
Циклы позволяют обрабатывать массивы, файлы, вычислять суммы, искать элементы.

Блок-схема цикла

с предусловием (while):



с параметром (for):



Цикл while в Python

Синтаксис:

```
while условие:
    # тело цикла
# обновление переменной
```

Пример: вывод чисел от 1 до 5

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

Важные особенности Python:

- Отступы обязательны! Тело цикла определяется отступами
- Двоеточие **:** после условия
- Нет фигурных скобок — в отличие от C++
- Обновление переменной должно быть внутри цикла, иначе — бесконечный цикл

Пример: сумма чисел от 1 до N

```
n = int(input("Введите N: "))
total = 0
i = 1

while i <= n:
    total += i
    i += 1

print(f"Сумма чисел от 1 до {n}: {total}")
```

Пример: бесконечный цикл с break

```
while True:
    command = input("Введите команду (exit для выхода): ")
    if command == "exit":
        break
    print(f"Вы ввели: {command}")
```

Опасность: если забыть обновить переменную — бесконечный цикл!

Цикл while в C++

Синтаксис:

```
while (условие) {
    // тело цикла
    // обновление переменной
}
```

Пример: вывод чисел от 1 до 5

```
#include <iostream>

int main() {
    int i = 1;
    while (i <= 5) {
        std::cout << i << "\n"; ++i;
    }
    return 0;
}
```

Цикл do-while (только в C++):

```
int i = 1;
do {
    std::cout << i << "\n";
    ++i;
} while (i <= 5);
```

Отличие: тело выполняется **хотя бы один раз**, потом проверяется условие.

Пример: сумма чисел от 1 до N

```
#include <iostream>

int main() {
    int n;
    std::cout << "Введите N: ";
    std::cin >> n;

    int total = 0;
    int i = 1;

    while (i <= n) {
        total += i;
        ++i;
    }

    std::cout << "Сумма чисел от 1 до " << n << ": " << total << "\n";
    return 0;
}
```

Особенности C++:

- Условие в круглых скобках **( )**
- Тело в фигурных скобках **{ }**
- Точка с запятой после инструкций
- Нет обязательных отступов (но нужны для читаемости)

Цикл for в Python

Синтаксис:

```
for переменная in последовательность:
# тело цикла
```

Пример: вывод чисел от 1 до 5

```
for i in range(1, 6):
    print(i)
```

Функция range():

Вызов	Результат
range(5)	0, 1, 2, 3, 4
range(1, 6)	1, 2, 3, 4, 5
range(0, 10, 2)	0, 2, 4, 6, 8
range(5, 0, -1)	5, 4, 3, 2, 1
range(len(list))	0, 1, 2, ..., len-1

Особенность Python:

for — это **итерация по последовательности**, а не цикл с счётчиком.

Итерация по коллекциям:

```
# По списку
fruits = ["яблоко", "банан", "вишня"]
for fruit in fruits:
    print(fruit)

# По строке
for char in "Python":
    print(char)

# По словарю
person = {"name": "Иван", "age": 25}
for key in person:
    print(f"{key}: {person[key]}")

# С enumerate (индекс + значение)
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")

# С zip (параллельная итерация)
names = ["Иван", "Мария"]
ages = [25, 30]
for name, age in zip(names, ages):
    print(f"{name}: {age}")
```

Цикл for в C++

Классический for:

```
for (инициализация; условие; инкремент) {
    // тело цикла
}
```

Пример: вывод чисел от 1 до 5

```
#include <iostream>

int main() {
    for (int i = 1; i <= 5; ++i) {
        std::cout << i << "\n";
    }
    return 0;
}
```

Сравнение с Python:

Python	C++
for i in range(1, 6):	for (int i = 1; i <= 5; ++i)
for fruit in fruits:	for (auto& fruit : fruits)

Range-based for (C++11):

```
// Синтаксис
for (тип переменная : контейнер) {
    // тело цикла
}

// Пример: итерация по массиву
int numbers[] = {10, 20, 30, 40, 50};
for (int num : numbers) {
    std::cout << num << "\n";
}

// Пример: итерация по строке
std::string text = "Python";
for (char c : text) {
    std::cout << c << "\n";
}

// Пример: с vector
#include <vector>
std::vector<std::string> fruits = {"яблоко", "банан", "вишня"};
for (const std::string& fruit : fruits) {
    std::cout << fruit << "\n";
}
```

Python vs C++: циклы

Аспект	Python	C++
while	while условие:	while (условие) { }
do-while	Нет	do { } while (условие);
for (по диапазону)	for i in range(1, 6):	for (int i = 1; i <= 5; ++i)
for (по коллекции)	for x in list:	for (auto& x : container)
break	break	break
continue	continue	continue
Отступы	Обязательны	Для читаемости

Ключевое различие:

Python for — итерация по последовательности.

C++ классический for — цикл с счётчиком.

Пример: сумма чисел от 1 до N

Python:

```python
n = int(input("Введите N: "))
total = 0

for i in range(1, n + 1):
    total += i

print(f"Сумма: {total}")
```

C++:

```cpp
int n;
std::cout << "Введите N: ";
std::cin >> n;

int total = 0;
for (int i = 1; i <= n; ++i) {
    total += i;
}

std::cout << "Сумма: " << total << "\n";
```

Управление циклом

Оператор break: выход из цикла

Что делает:

break немедленно прерывает выполнение цикла и передаёт управление следующей инструкции после цикла.

Python:

```python
# Поиск первого чётного числа
numbers = [1, 3, 5, 8, 9, 10]

for num in numbers:
    if num % 2 == 0:
        print(f"Первое чётное: {num}")
        break
# Вывод: Первое чётное: 8
```

Пример: выход по команде пользователя

```python
while True:
    command = input("Введите команду (exit для выхода): ")
    if command == "exit":
        print("Выход...")
        break
    print(f"Вы ввели: {command}")
```

C++:

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> numbers = {1, 3, 5, 8, 9, 10};

    for (int num : numbers) {
        if (num % 2 == 0) {
            std::cout << "Первое чётное: " << num << "\n";
            break;
        }
    }
    return 0;
}
```

Типичные применения:

- Поиск элемента — нашли, выходим
- Проверка условия — нарушено, выходим
- Меню с командой выхода

Оператор continue: пропуск итерации

Что делает:
continue немедленно переходит к следующей итерации цикла, пропуская оставшиеся инструкции тела.

Python:

```python
# Вывод только нечётных чисел
for i in range(1, 11):
    if i % 2 == 0:
        continue        # пропускаем чётные
    print(i)
# Вывод: 1, 3, 5, 7, 9
```

Пример: обработка данных с пропуском

```python
# Сумма только положительных чисел
numbers = [5, -3, 8, -1, 10, -7, 2]
total = 0

for num in numbers:
    if num < 0:
    continue            # пропускаем отрицательные
    total += num

print(f"Сумма положительных: {total}")        # 25
```

C++:

```cpp
#include <iostream>

int main() {
    for (int i = 1; i <= 10; ++i) {
        if (i % 2 == 0) {
            continue;                // пропускаем чётные
        }
        std::cout << i << "\n";
    }
    return 0;
}
```

Различие break и continue:

Оператор	Действие
break	Полностью выходит из цикла
continue	Пропускает текущую итерацию, переходит к следующей

pass (Python) и пустой оператор (C++)

Python - pass:

pass – это пустая инструкция. Она ничего не делает, но нужна там, где синтаксис требует наличия блока кода.

Python:

```python
# pass в цикле
for i in range(5):
    pass                # заглушка — ничего не делаем

# pass в условии
if x > 0:
    pass                # TODO: реализовать позже
else:
    print("x не положительное")

 # pass в функции
def my_function():
    pass                # TODO: реализовать
```

Применение:

- Заглушки для функций, которые будут реализованы позже
- Пустые блоки в условиях
- Синтаксические требования

C++ - пустой оператор ; :

```cpp
// Пустой цикл
for (int i = 0; i < 5; ++i) {
    ;     // пустой оператор
}

// Пустое условие
if (x > 0) {
    ;     // ничего не делаем
} else {
    std::cout << "x не положительное\n";
}

// Пустая функция
void myFunction() {
    // пустое тело
}
```

Вложенные циклы

Вложенные циклы — это циклы внутри других циклов. Внешний цикл выполняется один раз, внутренний — полностью для каждой итерации внешнего.

Python:

```python
# Таблица умножения
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i * j:4}", end="")
    print() # перевод строки после внутреннего цикла
```

C++:

```cpp
#include <iostream>
#include <iomanip>

int main() {
    for (int i = 1; i <= 5; ++i) {
        for (int j = 1; j <= 5; ++j) {
            std::cout << std::setw(4) << i * j;
        }
        std::cout << "\n";
    }
    return 0;
}
```

Пример: вывод «ёлочки»

Python:

```python
n = 5
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

C++:

```cpp
int n = 5;
for (int i = 1; i <= n; ++i) {
    for (int j = 0; j < n - i; ++j) {
        std::cout << " ";
    }
    for (int j = 0; j < 2 * i - 1; ++j) {
        std::cout << "*";
    }
    std::cout << "\n";
}
```

Совет: избегайте вложенности более 3 уровней — это ухудшает читаемость.

Практика

Задание 1: Табуляция функции

Задача: Вывести таблицу значений функции y = sin(x) для x от 0 до π с шагом π/10.

```python
import math

print(f"{'x':>8} {'sin(x)':>10}")
print("-" * 20)

for i in range(11):
    x = i * math.pi / 10
    y = math.sin(x)
    print(f"{x:8.4f} {y:10.4f}")
```

Ожидаемый результат:

```
       x     sin(x)
--------------------
  0.0000     0.0000
  0.3142     0.3090
  0.6283     0.5878
  ...
  3.1416     0.0000
```

Вариации:

- y = cos(x), y = tan(x)
- Другой диапазон и шаг
- Вывод в файл

```cpp
#include <iostream>
#include <cmath>
#include <iomanip>

int main() {
    std::cout << std::setw(8) << "x"
              << std::setw(10) << "sin(x)" << "\n";
    std::cout << std::string(20, '-') << "\n";

    std::cout << std::fixed << std::setprecision(4);

    for (int i = 0; i <= 10; ++i) {
        double x = i * M_PI / 10;
        double y = std::sin(x);
        std::cout << std::setw(8) << x
                  << std::setw(10) << y << "\n";
    }
    return 0;
}
```

Задание 2: Сумма и произведение чисел

Задача: Написать программу, которая вычисляет:

- Сумму чисел от 1 до N
- Произведение чисел от 1 до N (факториал)
- Сумму квадратов чисел от 1 до N

```python
n = int(input("Введите N: "))

sum_numbers = 0
product = 1
sum_squares = 0

for i in range(1, n + 1):
    sum_numbers += i
    product *= i
    sum_squares += i ** 2

print(f"Сумма: {sum_numbers}")
print(f"Произведение (N!): {product}")
print(f"Сумма квадратов: {sum_squares}")
```

Обратите внимание:

- В C++ используем long long для факториала — числа быстро растут
- В Python int неограничен, можно не беспокоиться

```cpp
#include <iostream>

int main() {
    int n;
    std::cout << "Введите N: ";
    std::cin >> n;

    long long sumNumbers = 0;
    long long product = 1;
    long long sumSquares = 0;

    for (int i = 1; i <= n; ++i) {
        sumNumbers += i;
        product *= i;
        sumSquares += i * i;
    }

    std::cout << "Сумма: " << sumNumbers << "\n";
    std::cout << "Произведение (N!): " << product << "\n";
    std::cout << "Сумма квадратов: " << sumSquares << "\n";

    return 0;
}
```

Задание 3: Числа Фибоначчи

Задача: Вывести первые N чисел Фибоначчи.

```python
n = int(input("Введите N: "))

a, b = 0, 1

print(f"Первые {n} чисел Фибоначчи:")
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b

print()
```

Пример работы (n = 10):
0 1 1 2 3 5 8 13 21 34

Рекурсивная версия (для сравнения):

```python
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

```cpp
#include <iostream>

int main() {
    int n;
    std::cout << "Введите N: ";
    std::cin >> n;

    long long a = 0, b = 1;

    std::cout << "Первые " << n << " чисел Фибоначчи:\n";
    for (int i = 0; i < n; ++i) {
        std::cout << a << " ";
        long long temp = a + b;
        a = b;
        b = temp;
    }
    std::cout << "\n";

    return 0;
}
```

Сравнение: итеративная версия быстрее, рекурсивная элегантнее.

Задание 4: Разложение на простые множители

Задача: Разложить число на простые множители.

```python
n = int(input("Введите число: "))
original = n
factors = []

divisor = 2
while divisor * divisor <= n:
    while n % divisor == 0:
        factors.append(divisor)
        n //= divisor
    divisor += 1

if n > 1:
    factors.append(n)

print(f"{original} = {' × '.join(map(str, factors))}")
```

Пример работы:

Введите число: 360
360 = 2 × 2 × 2 × 3 × 3 × 5

Разбор:

- Делим число на 2, пока делится
- Потом на 3, потом на 5 и т.д.
- Оптимизация: проверяем делители до √n

```cpp
#include <iostream>
#include <vector>

int main() {
    int n;
    std::cout << "Введите число: ";
    std::cin >> n;

    int original = n;
    std::vector<int> factors;

    for (int divisor = 2; divisor * divisor <= n; ++divisor) {
        while (n % divisor == 0) {
            factors.push_back(divisor);
            n /= divisor;
        }
    }

    if (n > 1) {
        factors.push_back(n);
    }

    std::cout << original << " = ";
    for (size_t i = 0; i < factors.size(); ++i) {
        if (i > 0) std::cout << " × ";
        std::cout << factors[i];
    }
    std::cout << "\n";

    return 0;
}
```

Задание 5: Обработка цифр числа

Задача: Написать программу, которая:

- Считает количество цифр в числе
- Находит сумму цифр
- Находит произведение цифр
- Переворачивает число

```python
n = int(input("Введите число: "))
original = n

count = 0
total = 0
product = 1
reversed_num = 0

while n > 0:
    digit = n % 10
    count += 1
    total += digit
    product *= digit
    reversed_num = reversed_num * 10 + digit
    n //= 10

print(f"Количество цифр: {count}")
print(f"Сумма цифр: {total}")
print(f"Произведение цифр: {product}")
print(f"Перевёрнутое число: {reversed_num}")
```

Пример работы (n = 1234):

Количество цифр: 4
Сумма цифр: 10
Произведение цифр: 24
Перевёрнутое число: 4321

```cpp
#include <iostream>

int main() {
    int n;
    std::cout << "Введите число: ";
    std::cin >> n;

    int original = n;
    int count = 0;
    int total = 0;
    int product = 1;
    int reversedNum = 0;

    while (n > 0) {
        int digit = n % 10;
        ++count;
        total += digit;
        product *= digit;
        reversedNum = reversedNum * 10 + digit;
        n /= 10;
    }

    std::cout << "Количество цифр: " << count << "\n";
    std::cout << "Сумма цифр: " << total << "\n";
    std::cout << "Произведение цифр: " << product << "\n";
    std::cout << "Перевёрнутое число: " << reversedNum << "\n";

    return 0;
}
```

Задание 6: Проверка палиндрома

Задача: Проверить, является ли число палиндромом (читается одинаково слева направо и справа налево).

```python
n = int(input("Введите число: "))
original = n
reversed_num = 0

temp = n
while temp > 0:
    digit = temp % 10
    reversed_num = reversed_num * 10 + digit
    temp //= 10

if original == reversed_num:
    print(f"{original} — палиндром")
else:
    print(f"{original} — не палиндром")
```

Альтернатива для строк:

```python
text = input("Введите строку: ")
if text == text[::-1]:
    print("Палиндром")
```

Примеры:

- 12321 — палиндром
- 12345 — не палиндром
- 1001 — палиндром

```cpp
#include <iostream>

int main() {
    int n;
    std::cout << "Введите число: ";
    std::cin >> n;

    int original = n;
    int reversedNum = 0;
    int temp = n;

    while (temp > 0) {
        int digit = temp % 10;
        reversedNum = reversedNum * 10 + digit;
        temp /= 10;
    }

    if (original == reversedNum) {
        std::cout << original << " — палиндром\n";
    } else {
        std::cout << original << " — не палиндром\n";
    }

    return 0;
}
```

Задание 7. Вывод фигур из символов

Задача: Вывести следующие фигуры:

1. Прямоугольник N×M
2. Треугольник высотой N
3. Ёлочка высотой N
4. Ромб высотой N

```python
n = 5

# Прямоугольник
print("Прямоугольник:")
for i in range(n):
    print("*" * n)

# Треугольник
print("\nТреугольник:")
for i in range(1, n + 1):
    print("*" * i)

# Ёлочка
print("\nЁлочка:")
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))

# Ромб
print("\nРомб:")
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
for i in range(n - 1, 0, -1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

Совет: используйте "*" * k в Python для повторения строки.

```cpp
#include <iostream>

int main() {
    int n = 5;

    // Прямоугольник
    std::cout << "Прямоугольник:\n";
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) std::cout << "*";
        std::cout << "\n";
    }

    // Треугольник
    std::cout << "\nТреугольник:\n";
    for (int i = 1; i <= n; ++i) {
        for (int j = 0; j < i; ++j) std::cout << "*";
        std::cout << "\n";
    }

    // Ёлочка
    std::cout << "\nЁлочка:\n";
    for (int i = 1; i <= n; ++i) {
        for (int j = 0; j < n - i; ++j) std::cout << " ";
        for (int j = 0; j < 2 * i - 1; ++j) std::cout << "*";
        std::cout << "\n";
    }

    // Ромб
    std::cout << "\nРомб:\n";
    for (int i = 1; i <= n; ++i) {
        for (int j = 0; j < n - i; ++j) std::cout << " ";
        for (int j = 0; j < 2 * i - 1; ++j) std::cout << "*";
        std::cout << "\n";
    }
    for (int i = n - 1; i >= 1; --i) {
        for (int j = 0; j < n - i; ++j) std::cout << " ";
        for (int j = 0; j < 2 * i - 1; ++j) std::cout << "*";
        std::cout << "\n";
    }

    return 0;
}
```

Задание 8. Работа в мини-группах

Формат работы: группы по 3-4 человека.
Задача: Каждая группа получает свой вариант. Необходимо реализовать программу на **двух языках** (Python и C++) и сравнить результаты.

Варианты задач для групп:

Вариант	Задача
1	Вычисление НОД двух чисел через алгоритм Евклида
2	Проверка, является ли число совершенным
3	Вывод таблицы значений функции $y = x^2 - 2x + 1$ для x от −5 до 5
4	Нахождение всех простых чисел до N (решето Эратосфена)
5	Вычисление числа π методом Монте-Карло
6	Перевод числа из десятичной системы в двоичную и обратно
7	Вычисление суммы ряда 1 + 1/2 + 1/3 + ... + 1/N
8	Вывод треугольника Паскаля высотой N

Критерии оценки:

- Корректность решения на двух языках
- Понимание различий между языками
- Обработка краевых случаев
- Качество форматирования вывода

Презентация результатов: 2–3 минуты на группу.

Требования к отчёту и критерии оценки

Структура отчёта:

1. Титульный лист
2. Цель работы
3. Выполнение заданий:
   - Листинг кода для каждого задания
   - Скриншоты результатов
   - Примеры использования break, continue, pass
   - Результаты групповой работы
4. Ответы на контрольные вопросы
5. Вывод

Критерии оценки:

Критерий	Мин. балл	Макс. балл
Самостоятельная проработка теории, ответы на вопросы	1	2
Корректность выполнения заданий, рабочий код	3	6
Качество кода (PEP8, структура, комментарии)	1	2
ИТОГО	**5**	**10**

Формат сдачи: отчёт в формате PDF или DOCX + код в Git-репозитории (не менее 3 коммитов).

Контрольные вопросы

Контрольные вопросы

Вопросы для самопроверки:

1. **Что такое цикл?** Какие виды циклов вы знаете?
2. **В чём разница между while и for?** Когда какой использовать?
3. **Что делает оператор break?** А continue? В чём разница?
4. **Что такое pass в Python?** Зачем он нужен?
5. **Что такое вложенные циклы?** Сколько итераций выполнится для двух вложенных циклов от 1 до 5?
6. **Что такое бесконечный цикл?** Как его избежать?
7. **Как в Python сгенерировать последовательность чисел?** Что делает range(1, 10, 2)?
8. **В чём разница между классическим for и range-based for в C++?**



Ответы должны быть в отчёте.

Задания для самостоятельной работы

Задания для самостоятельной работы

Дополнительные задания (для продвинутых):

1. **Игра «Быки и коровы»:** Компьютер загадывает 4-значное число. Пользователь угадывает. Компьютер сообщает количество «быков» (цифра на своём месте) и «коров» (цифра есть, но не на своём месте).

2. **Вычисление числа e:** e = 1 + 1/1! + 1/2! + 1/3! + ... Вычислить с заданной точностью.

3. **Сортировка пузырьком:** Реализовать алгоритм сортировки массива.

4. **Бинарный поиск:** Реализовать алгоритм поиска элемента в отсортированном массиве.

5. **Сравнение языков (*):** Напишите одинаковую программу на Python и C++ и сравните:
   - Количество строк кода
   - Читаемость
   - Время выполнения.



Дополнительно:
- Создать README.md в репозитории

Спасибо
за внимание

